approximations and stale data (certainly not for older than a few seconds).

The other open question is: what makes data stale and do we have a quick way to know when it happens? If data simply ages, we can store a timestamp when we write it to cache, and check it when we read: if it’s too old to still be relevant, we can recompute and update the cache. If there are other external conditions, for instance a change in configuration or some event (like a new order made or a change of the required precision for calculus), we are still fine as long as we can check these conditions, and the check is relatively inexpensive, with respect to computing the value again from scratch.

In the rest of the chapter we simplify the problem by assuming that cached content doesn’t go stale. Handling stale content can be seen as an orthogonal task, that can enhance the base caching mechanism we built, for the most part independently of the choices we made for this mechanism.

7.2.3 Handling Asynchronous Calls

Acknowledging the existence of stale data and discussing workarounds was a good starting point towards generalizing caching solutions to more than just our carefully crafted example. The next step, is generalizing the computation model. So far, we have assumed we are in a single-threaded environment so calls are handled by a synchronous queue and are executed one after the other. This is not only